

Randomized Consensus

Mohsen Lesani

FLP Theorem

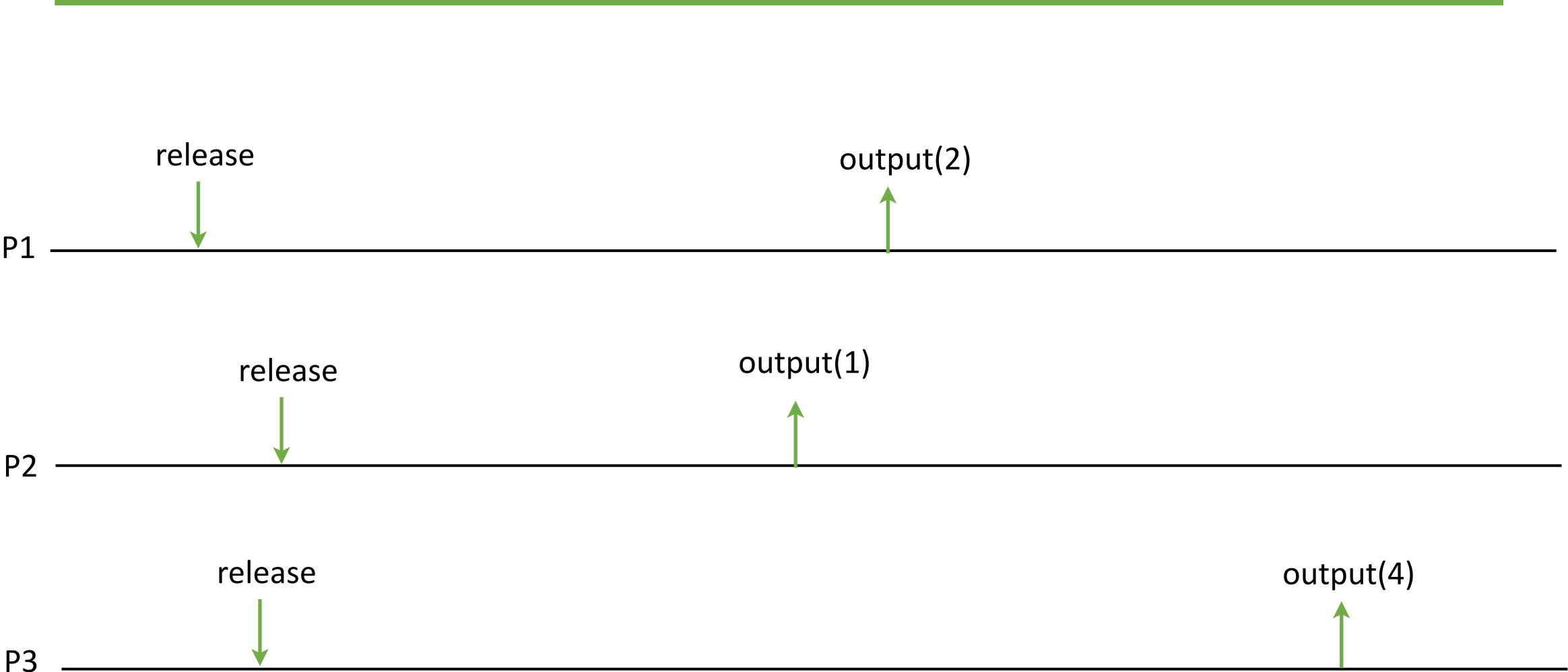
There is no deterministic consensus protocol for asynchronous distributed systems when even one process crashes.

Common Coin

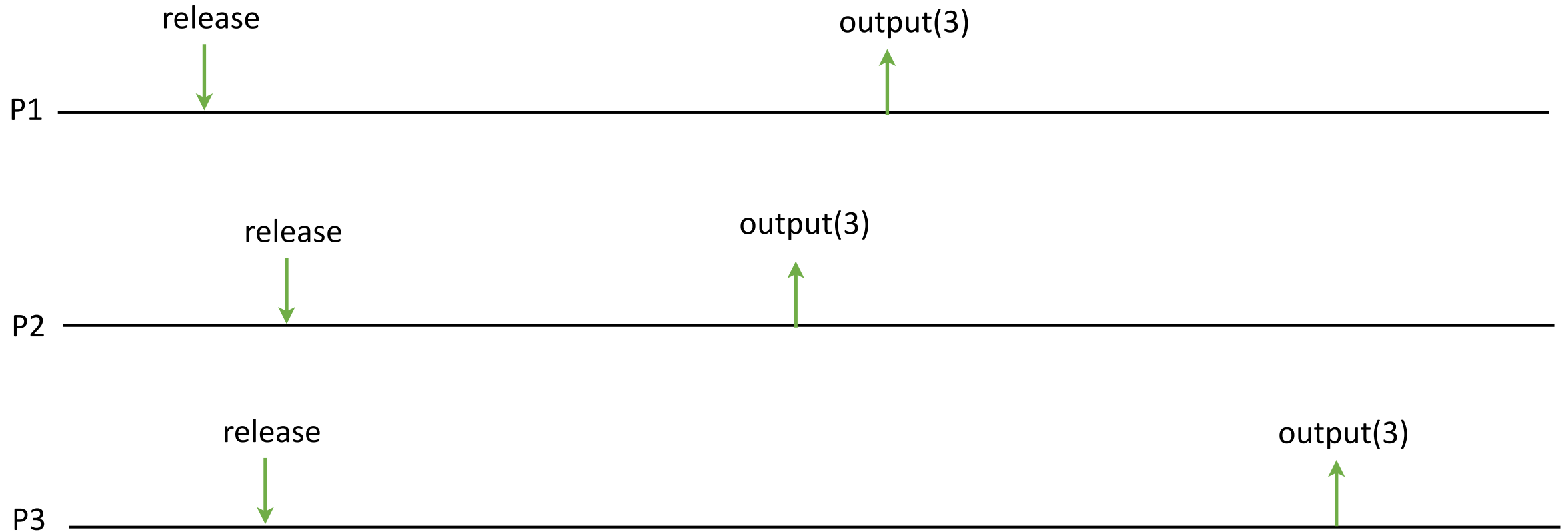
Random values from a domain D

- Request: $\langle \text{coin}, \text{release} \rangle$:
Releases the coin.
- Indication: $\langle \text{coin}, \text{output}(v) \rangle$:
Outputs the coin value $v \in D$.

Common Coin Example



Common Coin Example, Eventually



We say that the coin eventually matches.

Common Coin Specification

- Matching:
With probability at least δ , every correct process outputs the same coin value.
- No bias:
The distribution of the coin for the matched values is uniform over D (i.e., a matching coin outputs any value in D with probability $1/|D|$).
- Termination:
Every correct process eventually outputs a coin value.
- Other properties ...

Common Coin Implementation

Independent Choice Coin:

Upon releasing the coin, every process simply outputs a value locally at random from D according to the uniform distribution.

If the domain is one bit then this realizes a $\delta = 2^{-N+1}$ matching common coin, because the probability that every process selects some b is 2^{-N} , for two bits $b = \{0, 1\}$.

Common Coin Implementation

Beacon Coin:

An external trusted process, called the beacon, periodically chooses an unpredictable random value and broadcasts it at predefined times.

When an algorithm uses a sequence of common coin abstractions, then for the k -th coin, every process outputs the k -th random value received from the beacon.

This coin always matches but it's difficult to use it in an asynchronous system.

Randomized Consensus

- Agreement
- Integrity
- Validity
- **Probabilistic Termination:**
Every correct process eventually decides some value with **probability 1**.

Observation

The common coin will eventually output the same value for all processes.

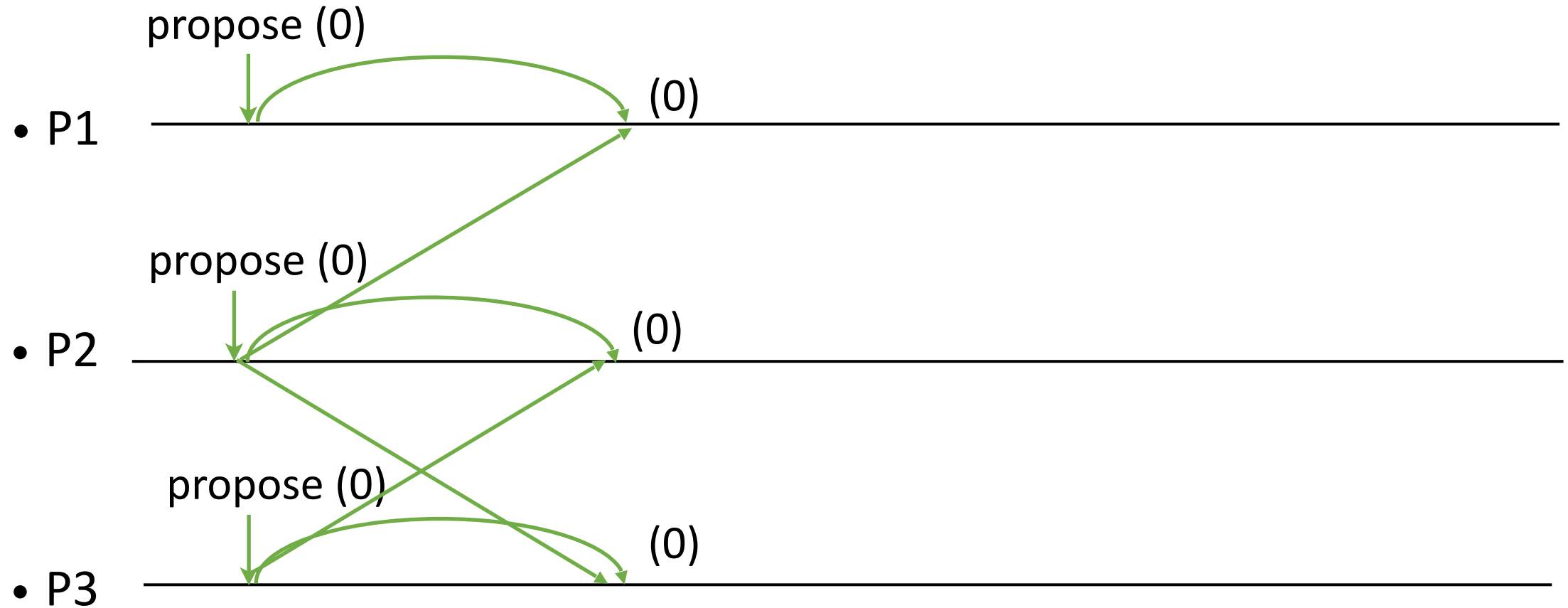
We need to design a consensus protocol that decides when all processes propose the same value.

Broadcast and wait for a quorum

Eventually all the proposals are the same.

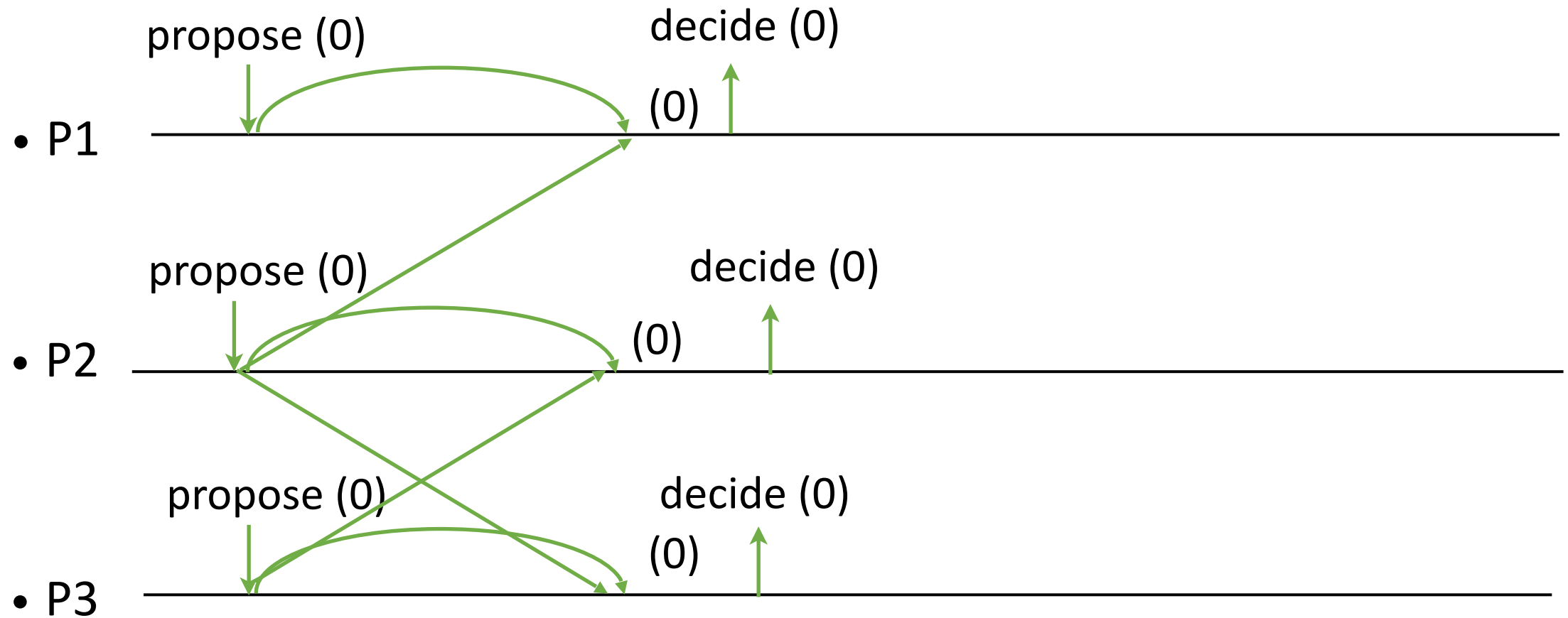
Broadcast and wait for a quorum

Eventually all the proposals are the same.



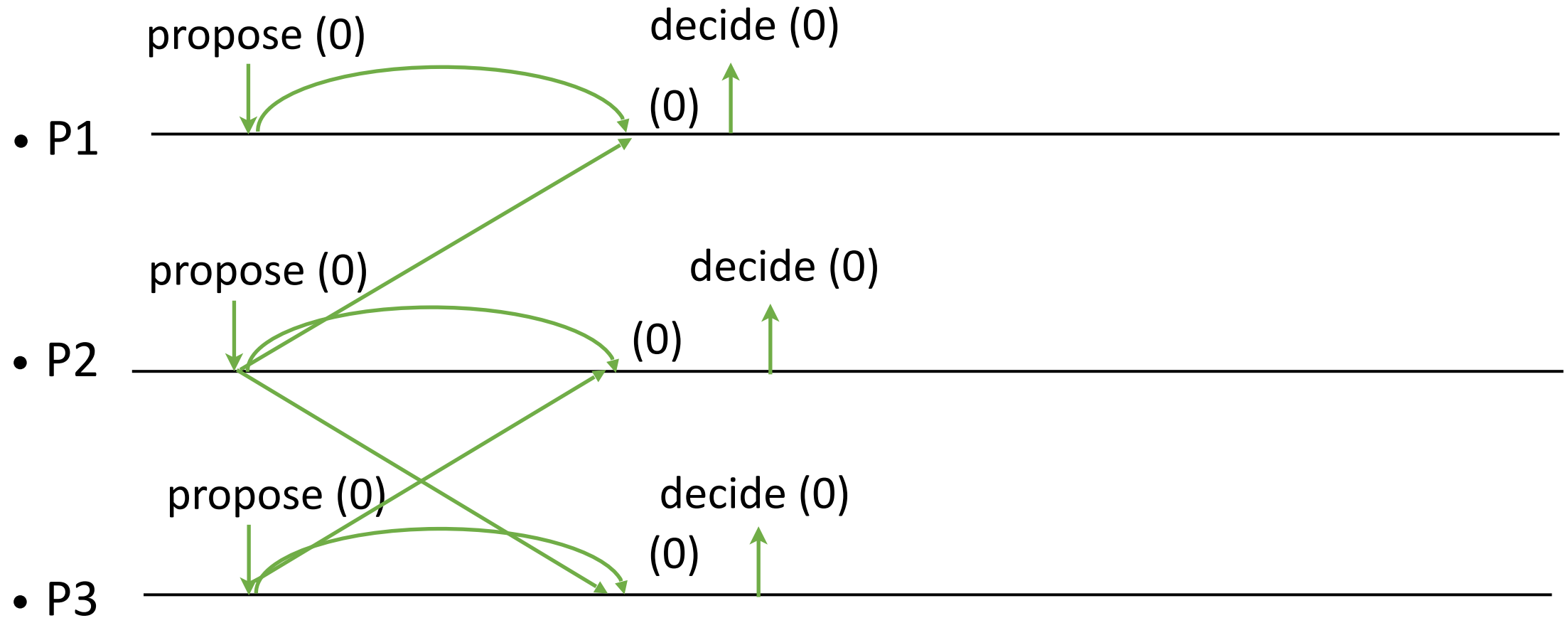
Broadcast and wait for a quorum

Eventually all the proposals are the same.



Broadcast and wait for a quorum

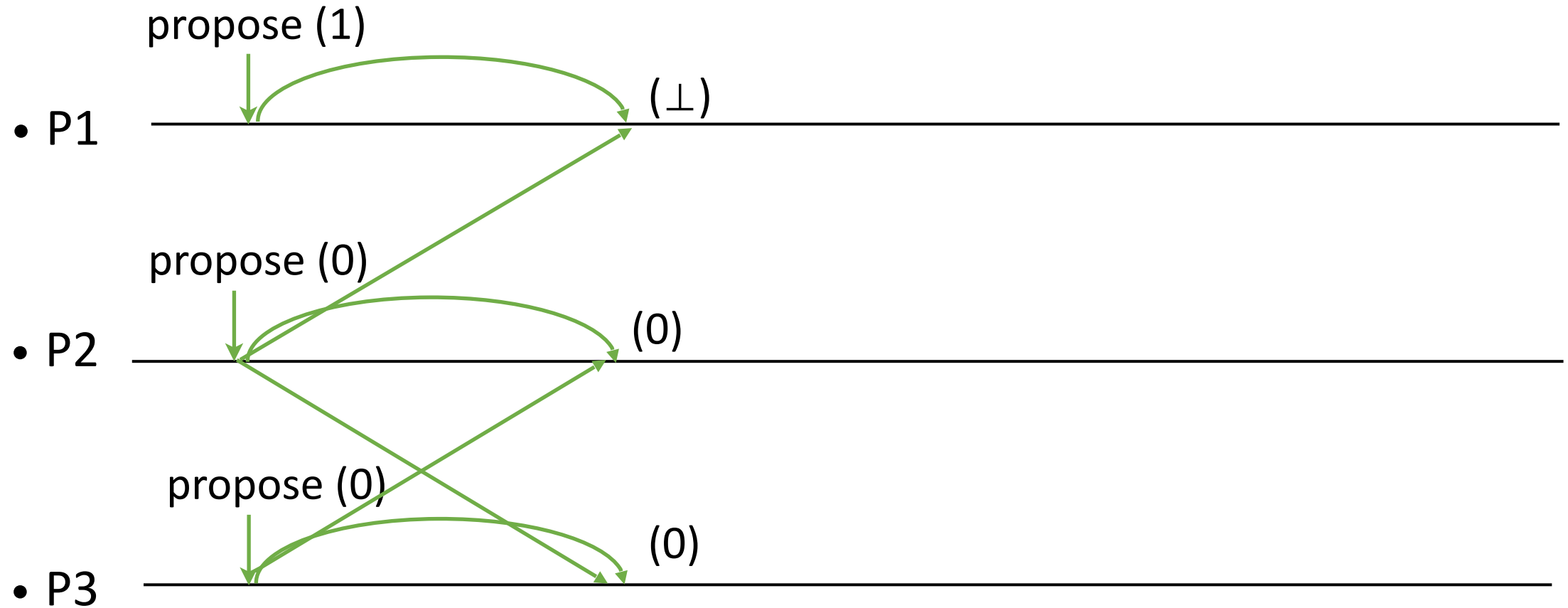
Eventually all the proposals are the same.



Does it have safety and liveness when proposals are different?

Broadcast and wait for a quorum

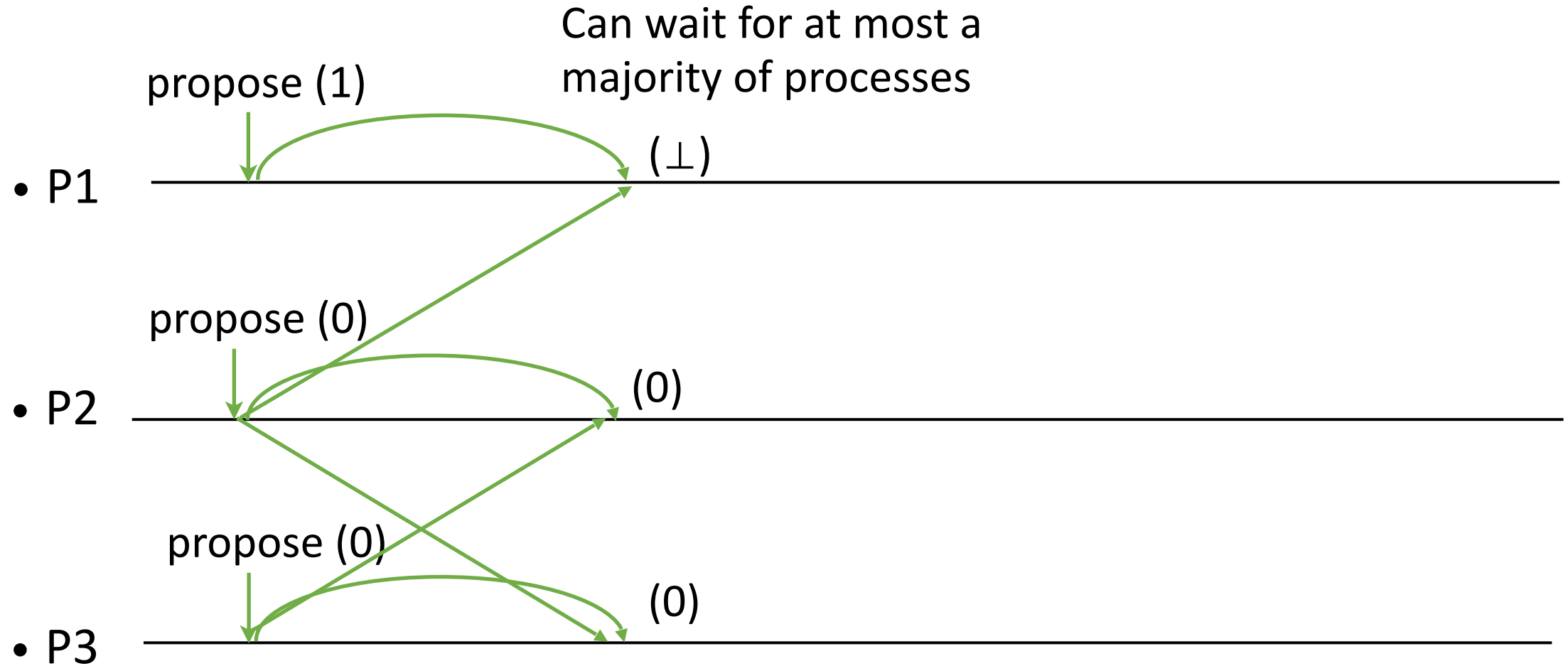
The proposals are different.



Broadcast and wait for a quorum

The proposals are different.

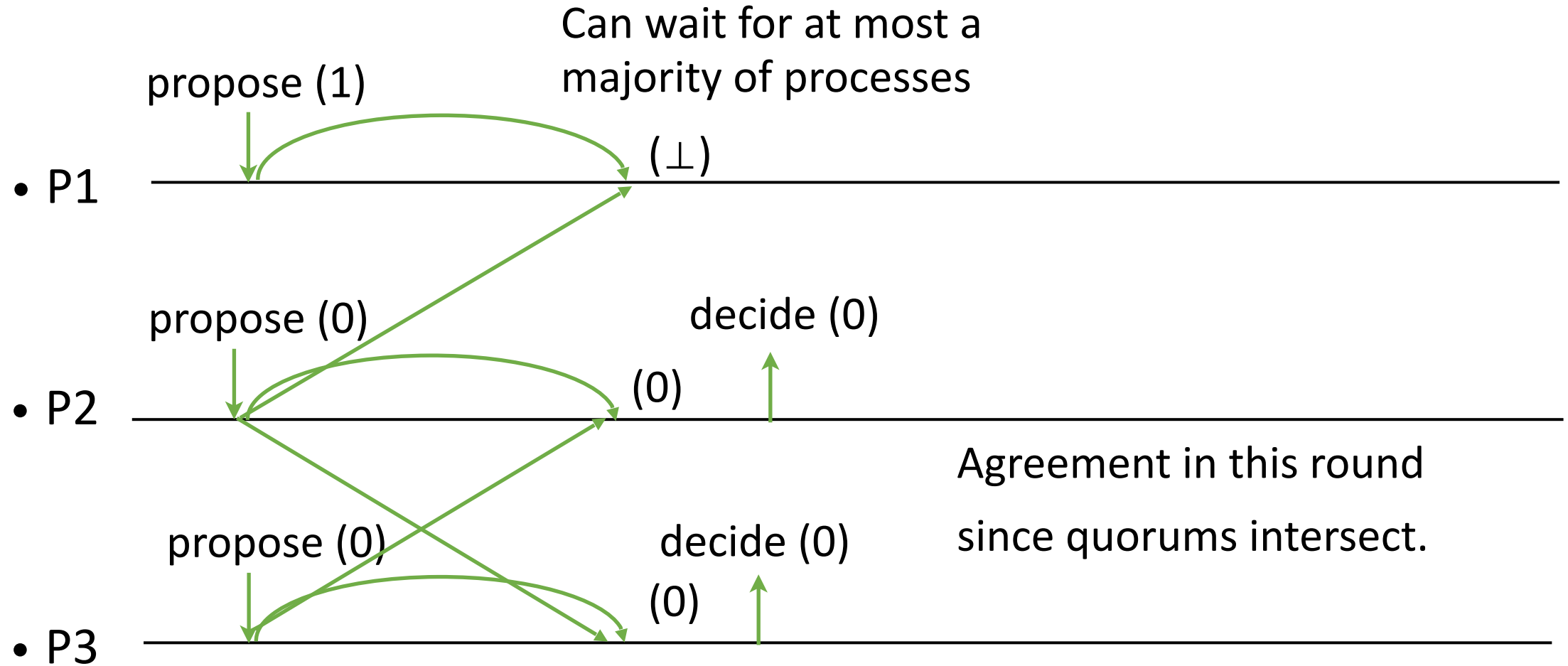
A majority (quorum) of processes are correct.



Broadcast and wait for a quorum

The proposals are different.

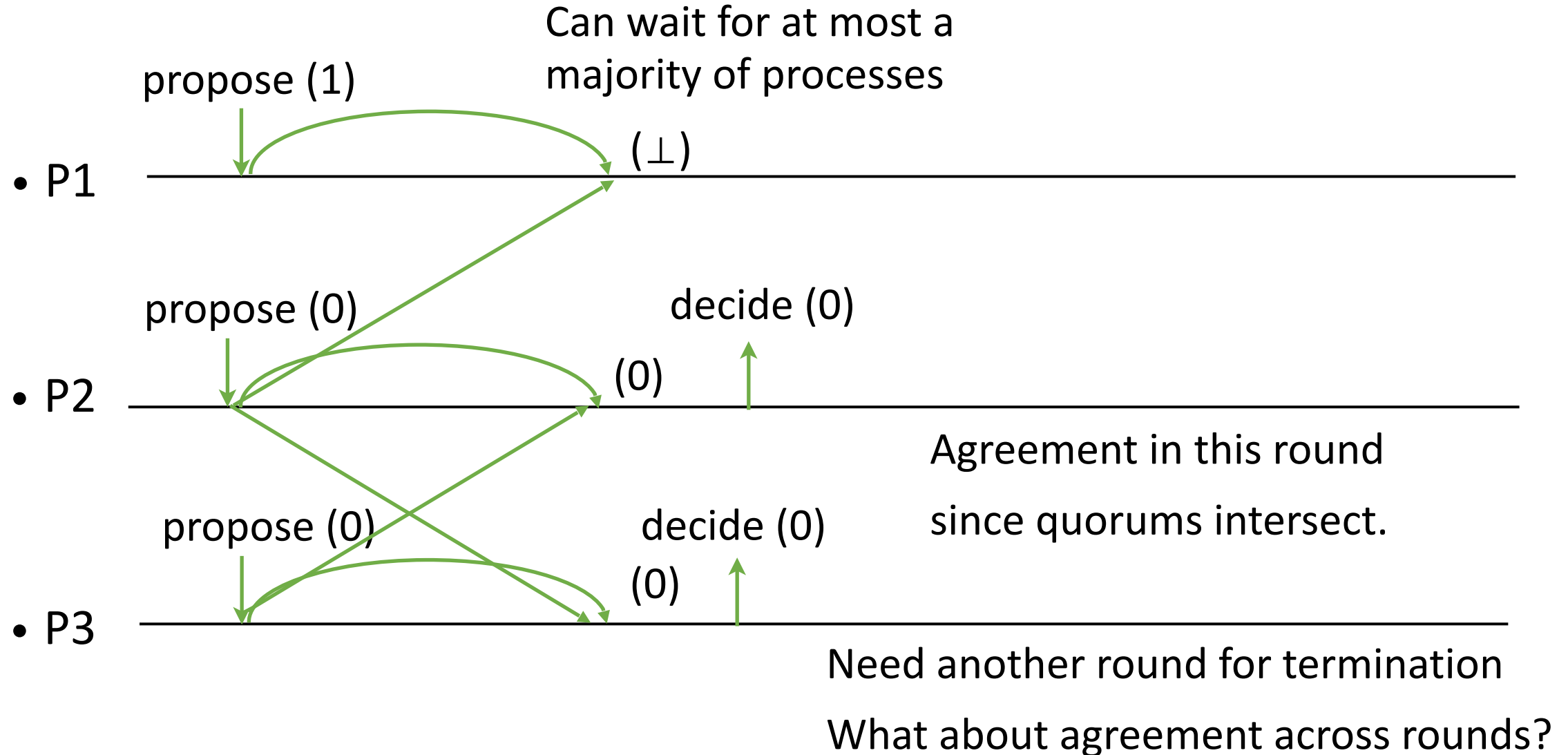
A majority (quorum) of processes are correct.



Broadcast and wait for a quorum

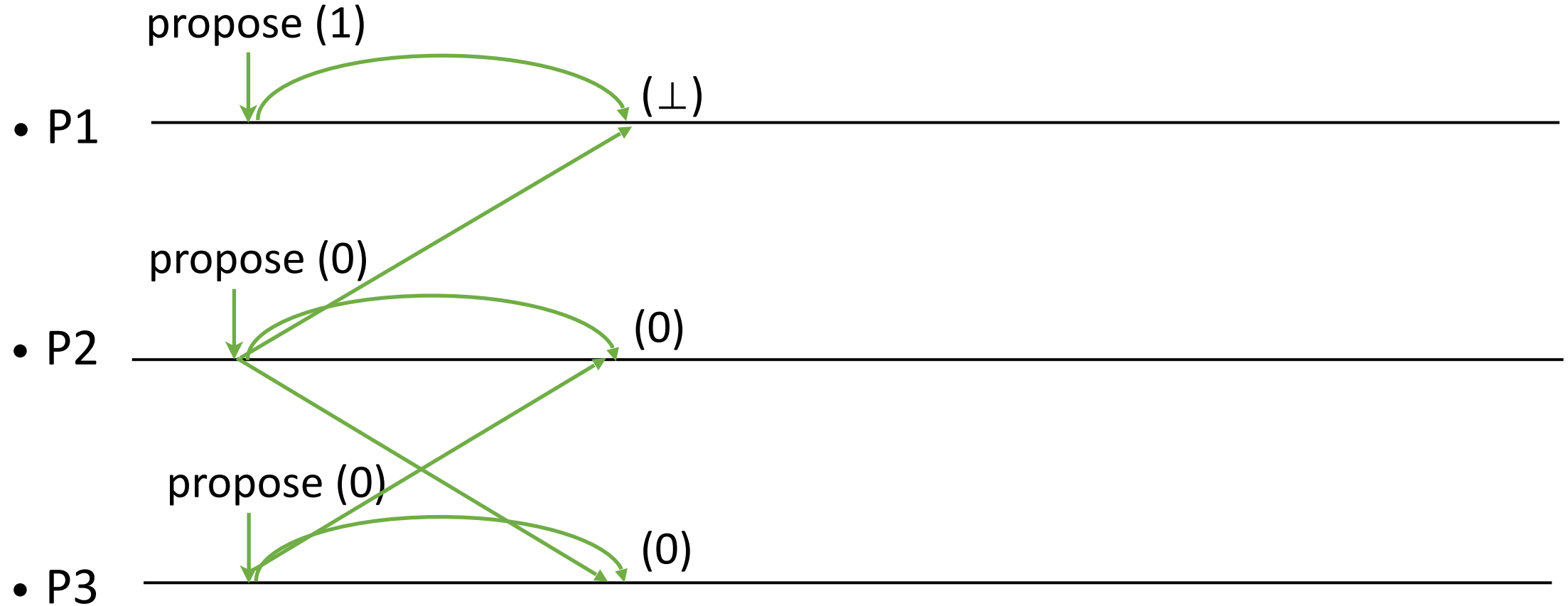
The proposals are different.

A majority (quorum) of processes are correct.



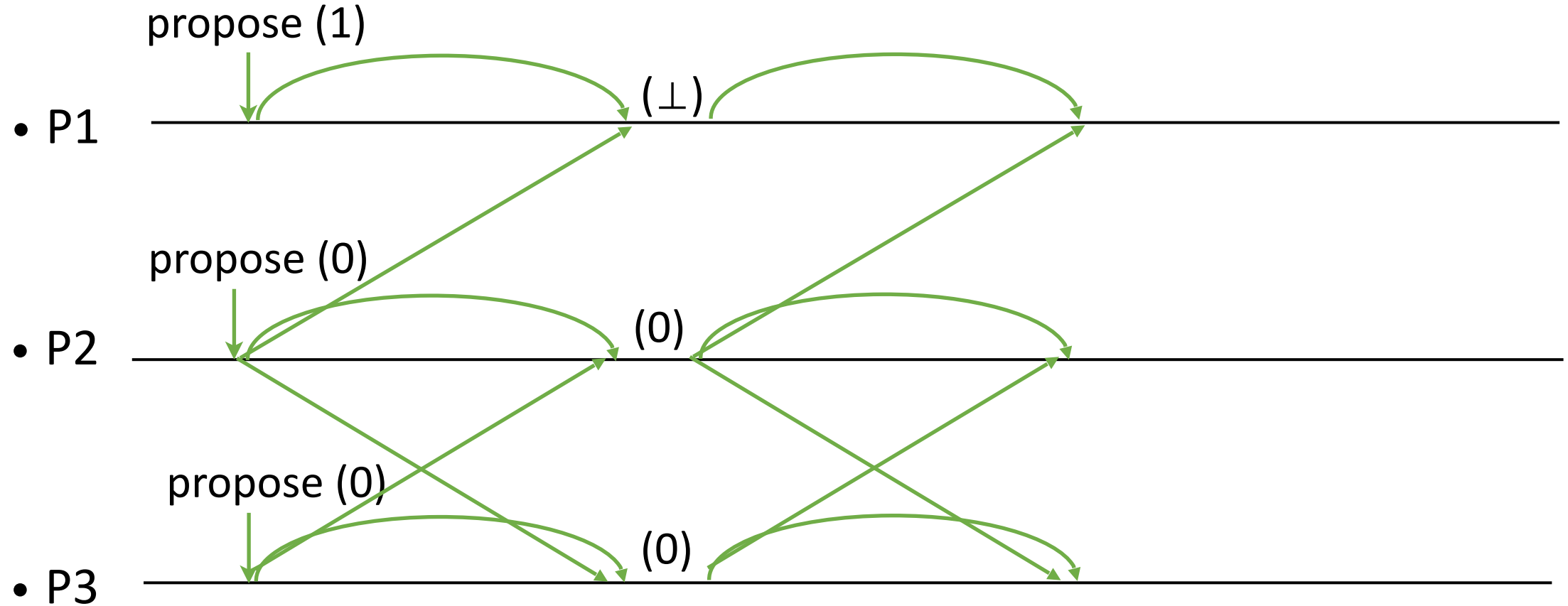
Adopt the decided value for next round

Second phase to make others adopt



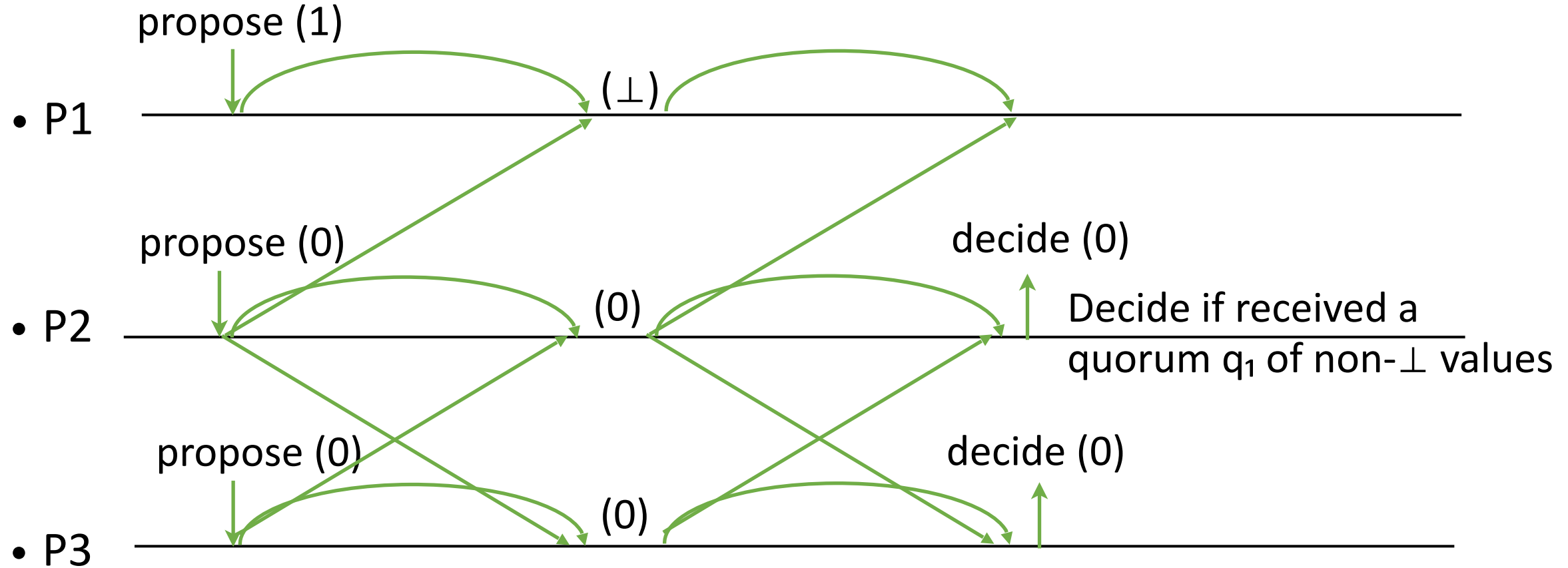
Adopt the decided value for next round

Second phase to make others adopt



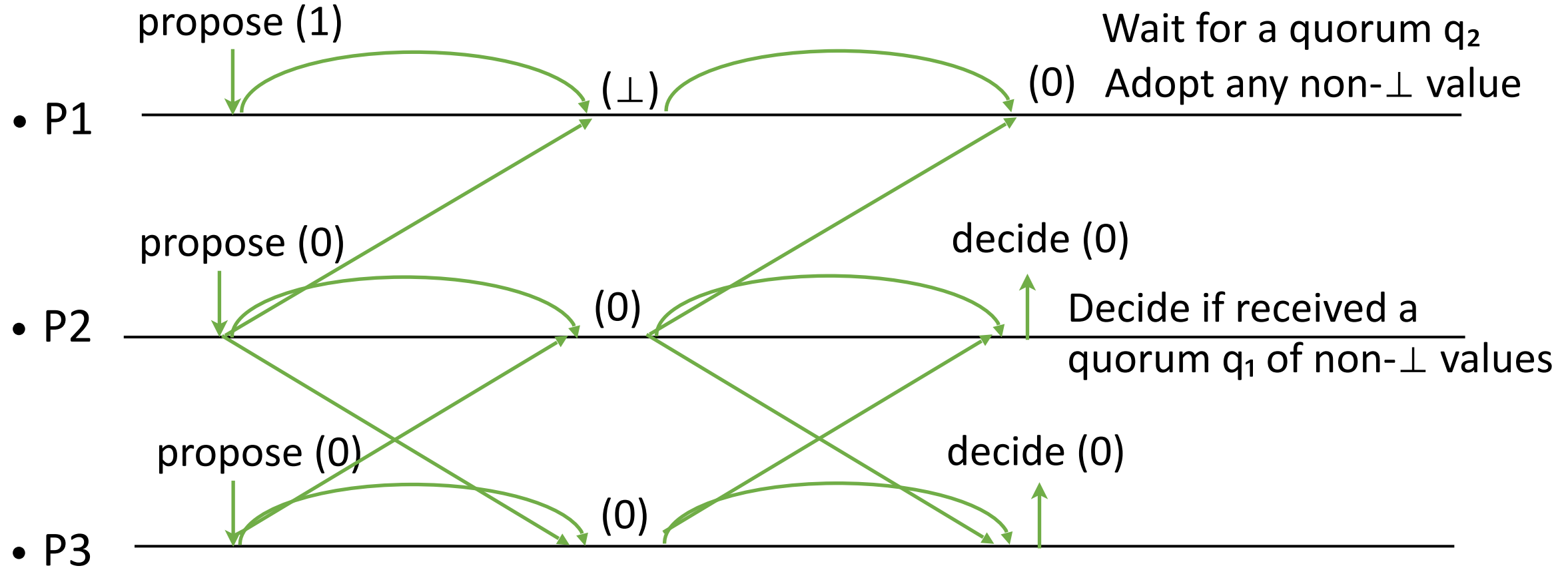
Adopt the decided value for next round

Second phase to make others adopt



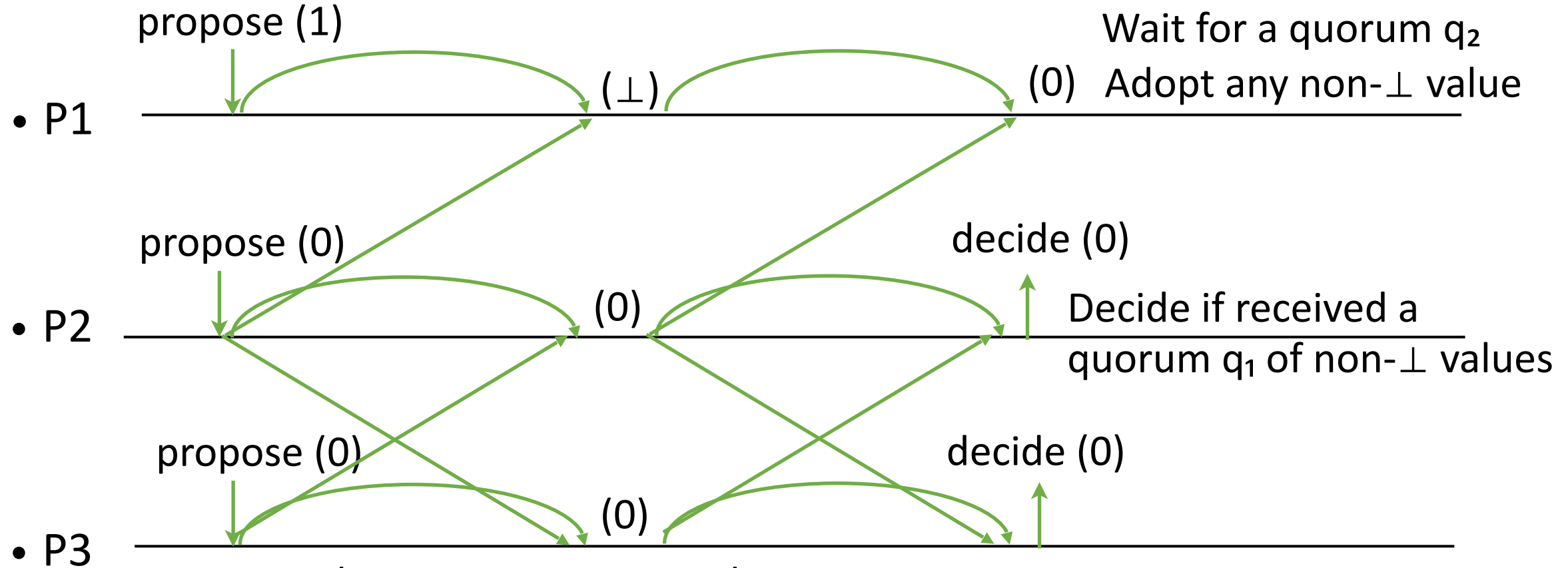
Adopt the decided value for next round

Second phase to make others adopt



Adopt the decided value for next round

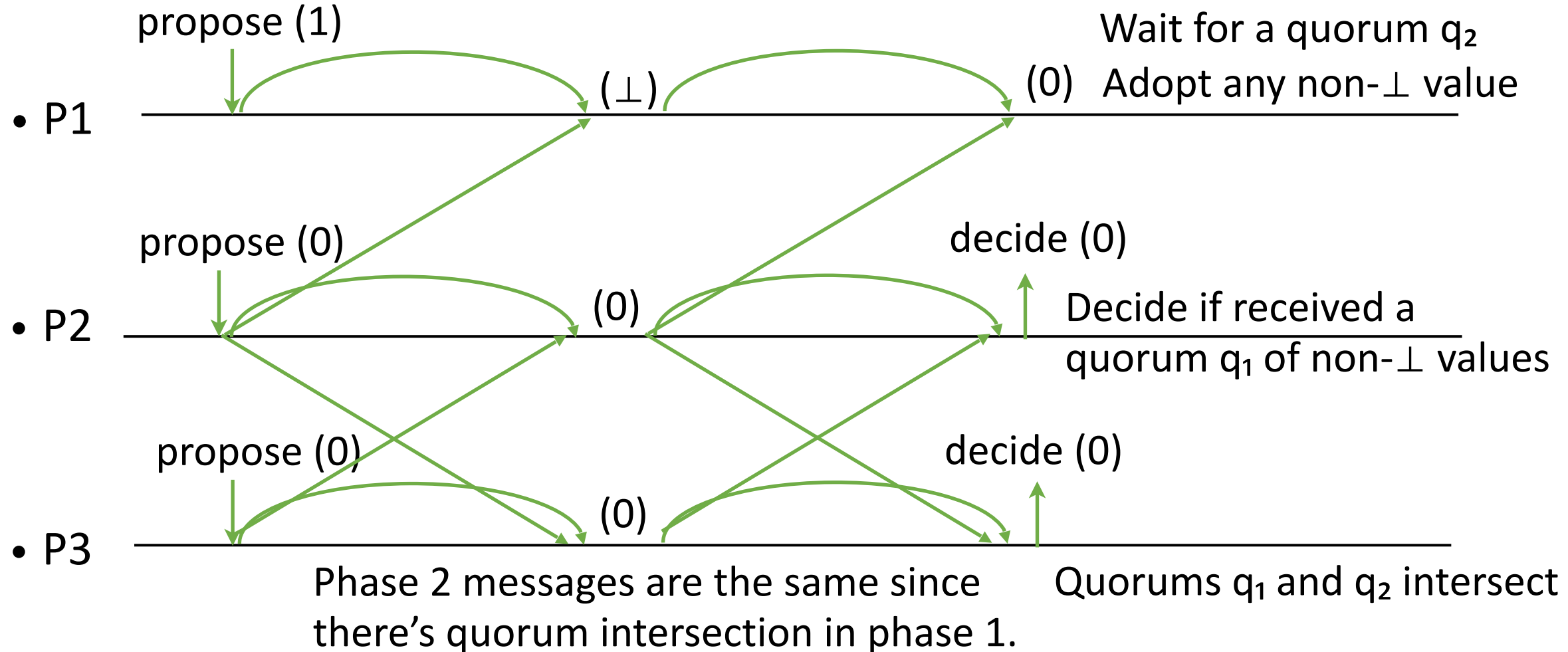
Second phase to make others adopt



Phase 2 messages are the same since there's quorum intersection in phase 1.

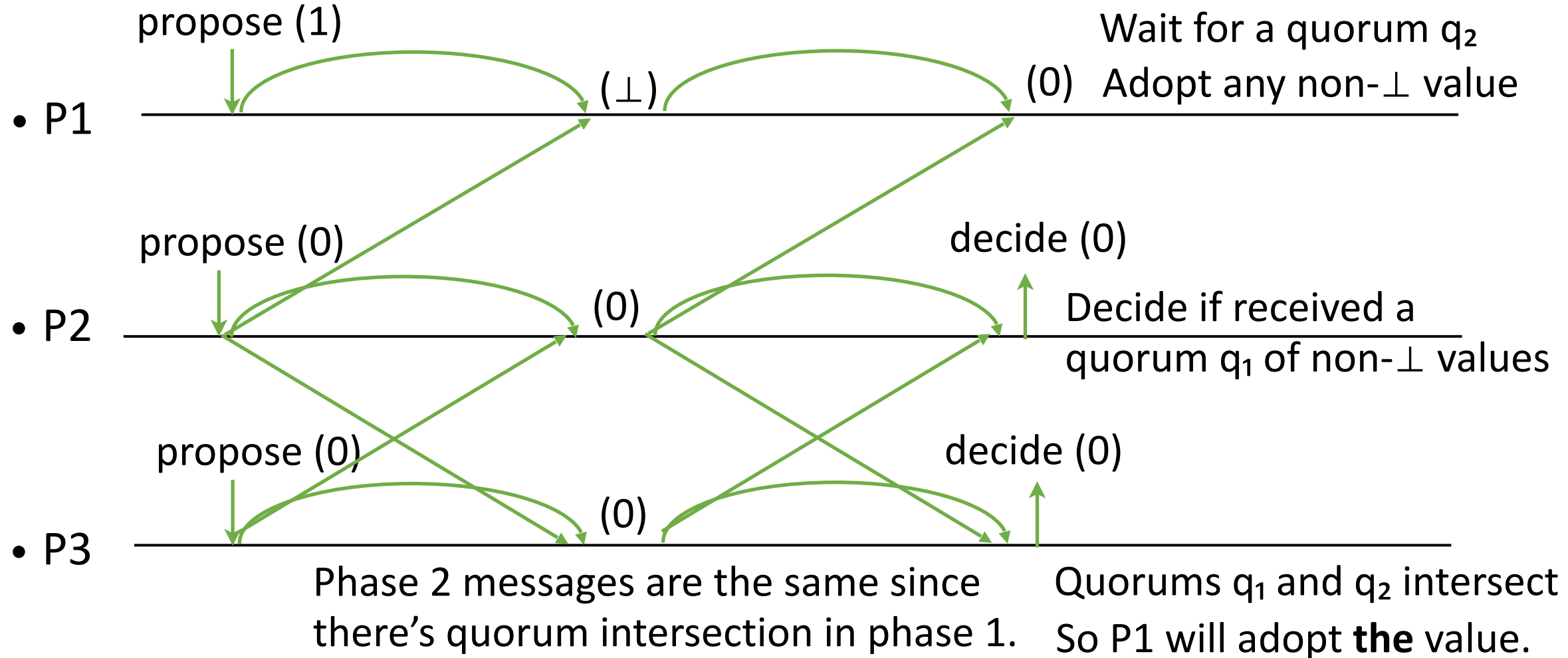
Adopt the decided value for next round

Second phase to make others adopt

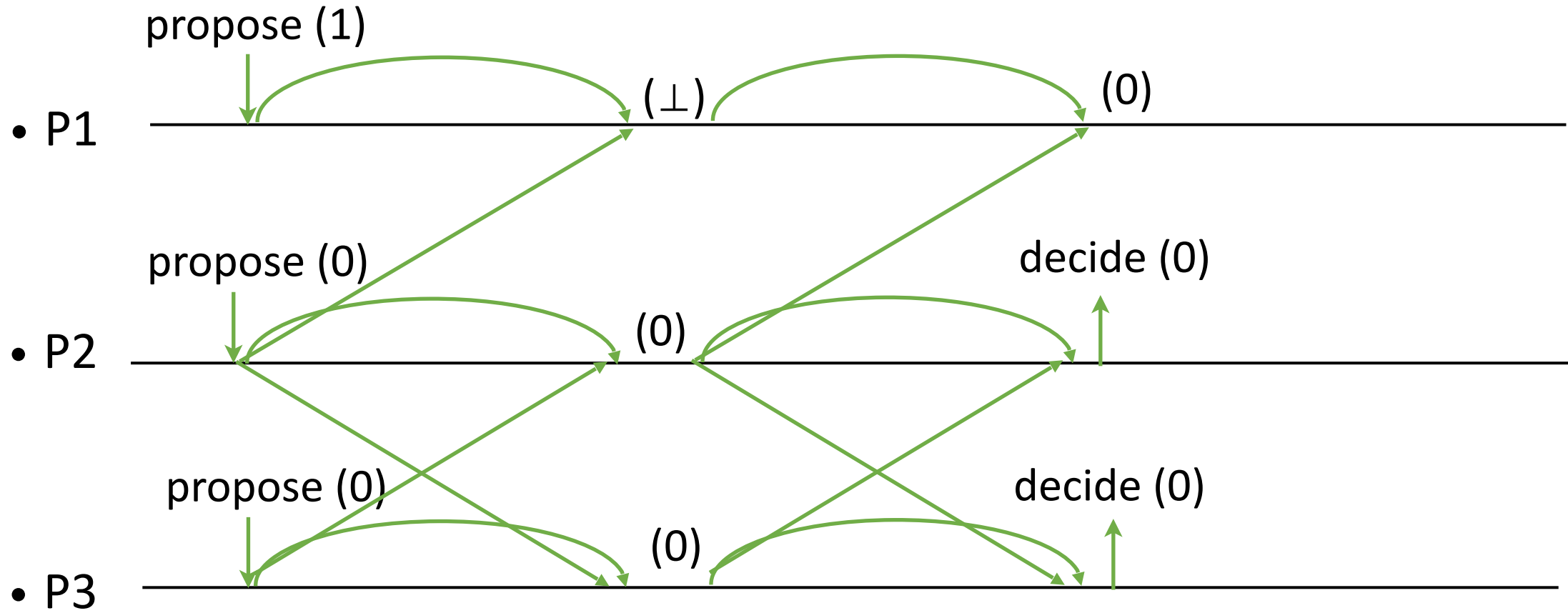


Adopt the decided value for next round

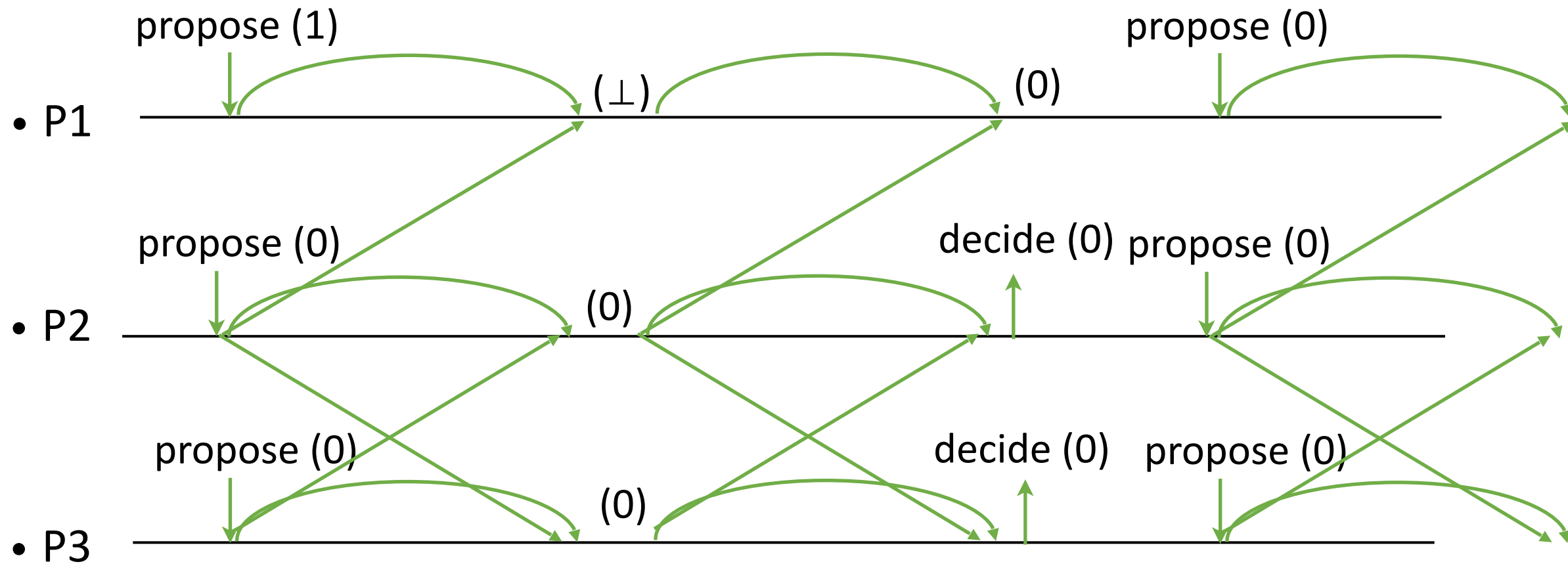
Second phase to make others adopt



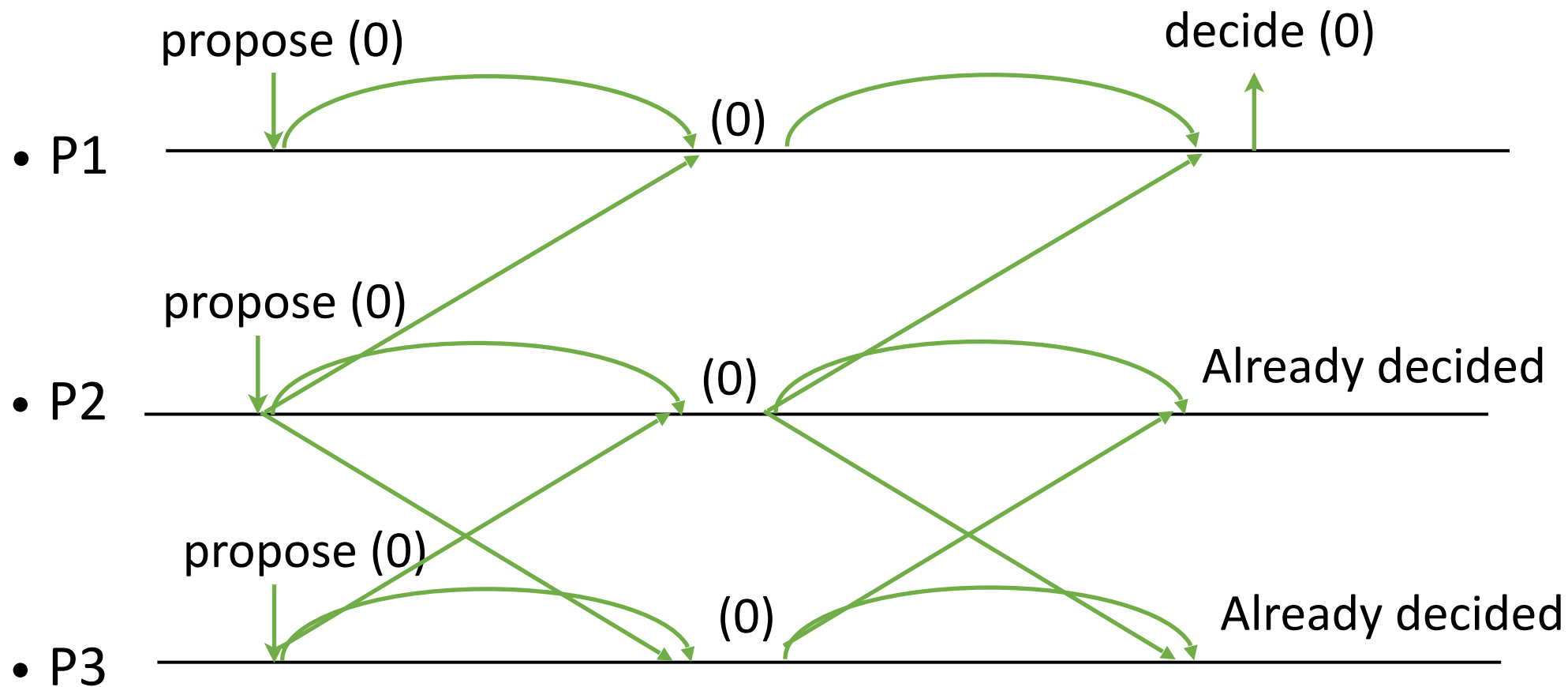
Next round



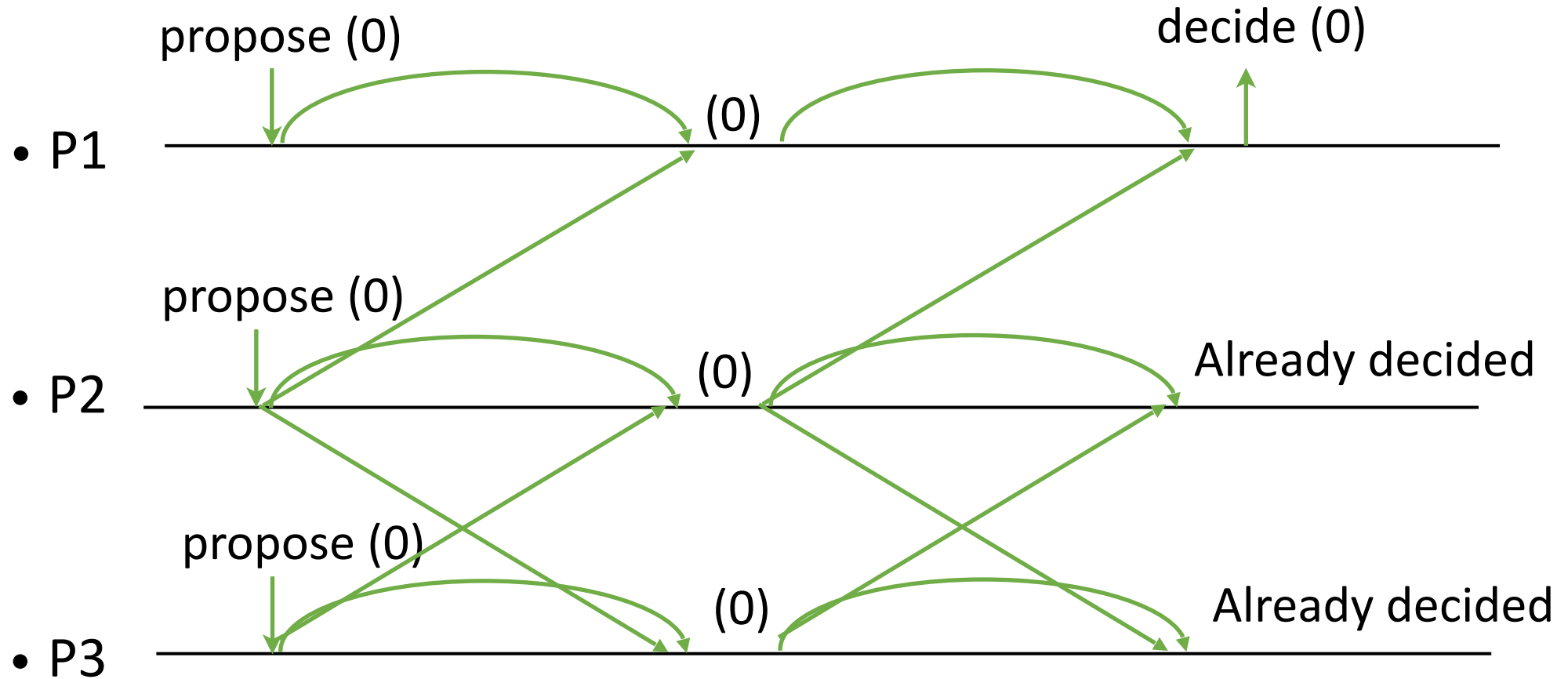
Next round



Next round

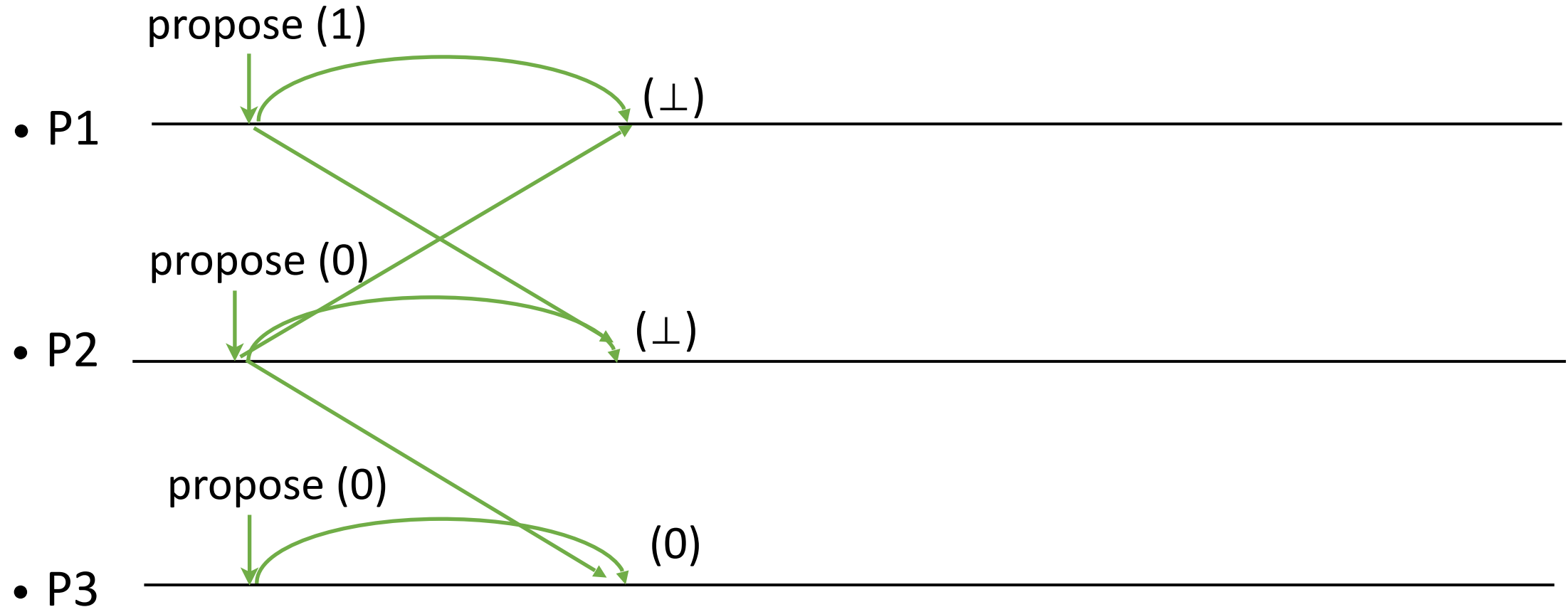


Next round

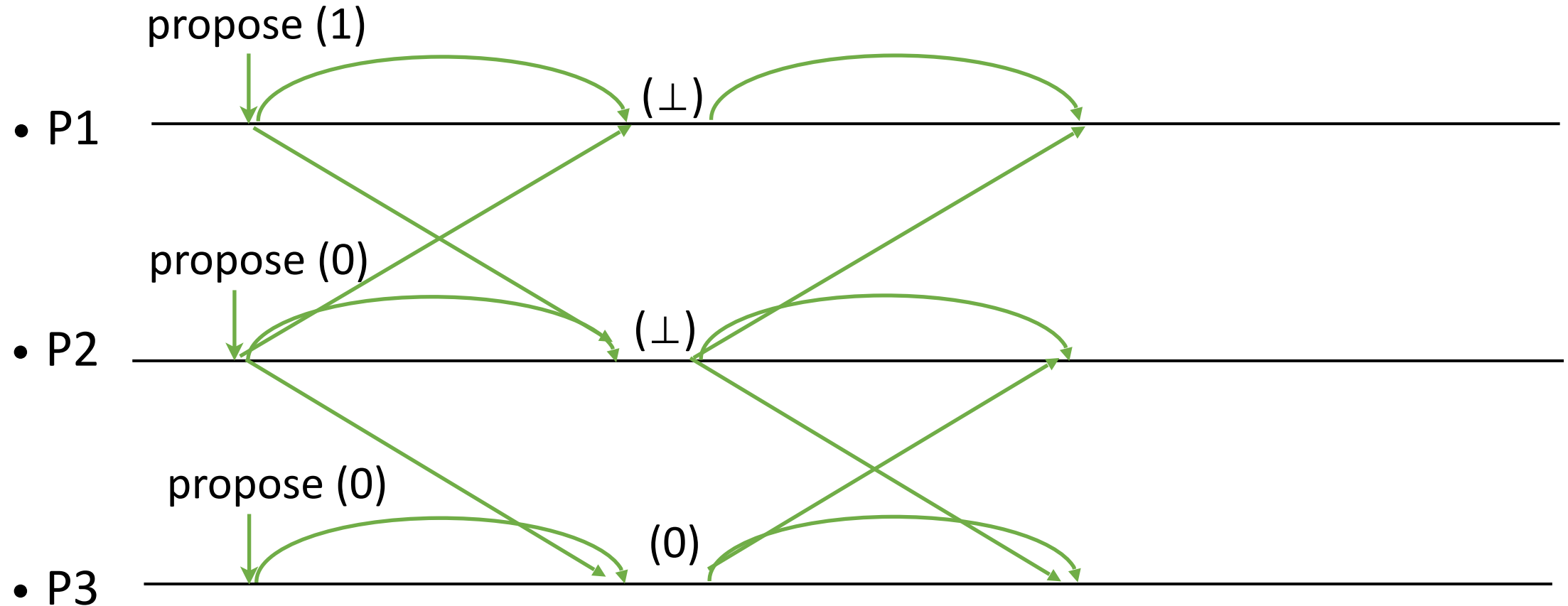


In this next round, all the remaining correct processes decide.

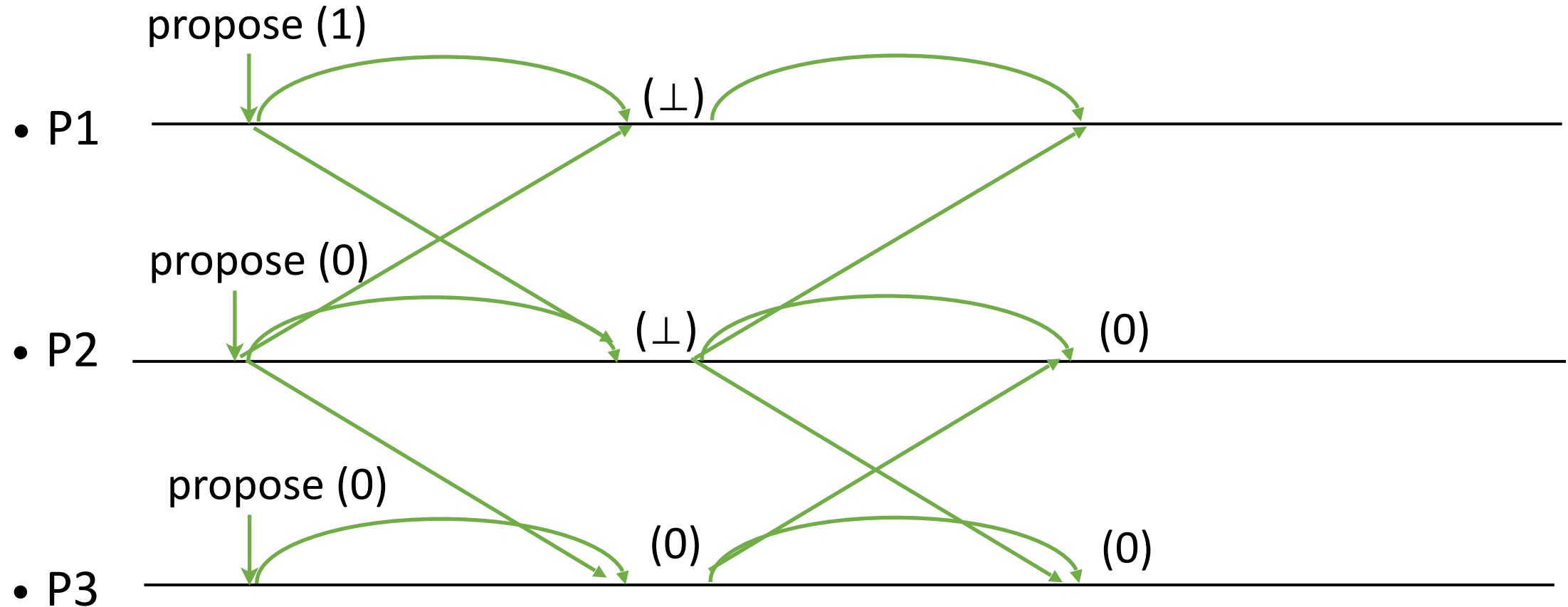
If not received non- $\perp$ value, get value from the coin for the next round



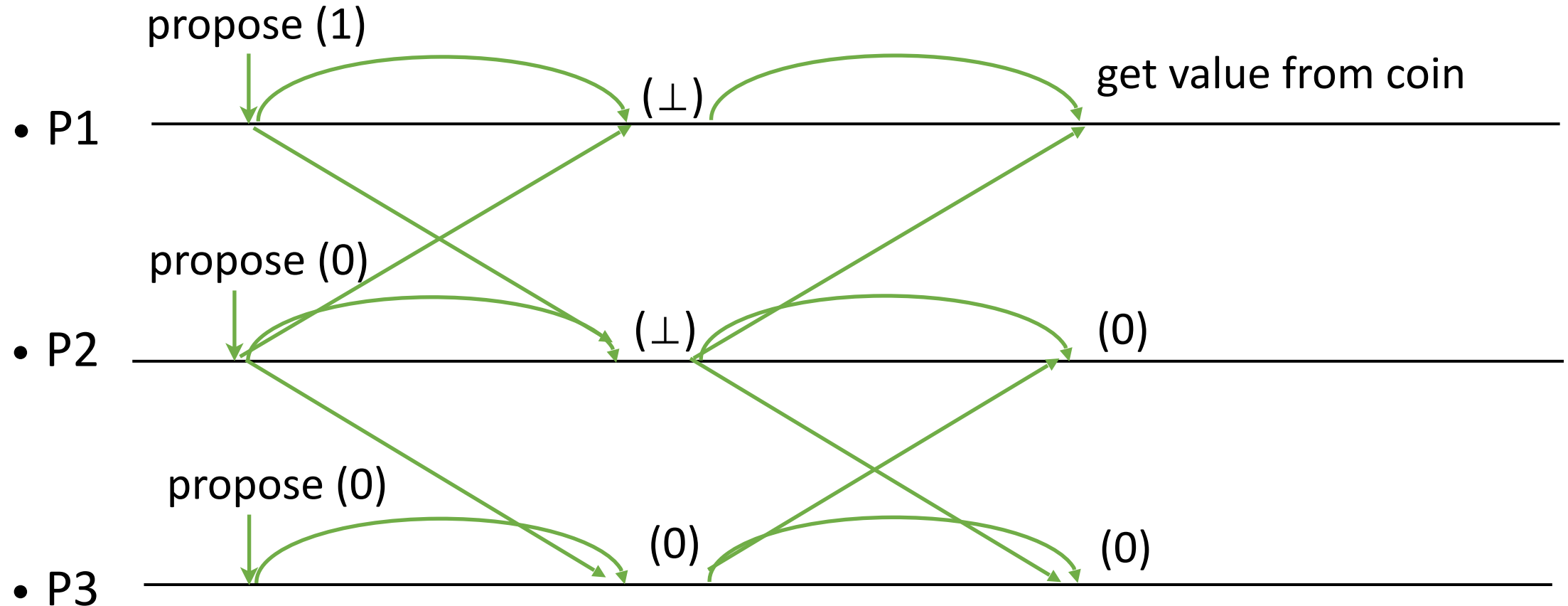
If not received non- $\perp$ value, get value from the coin for the next round



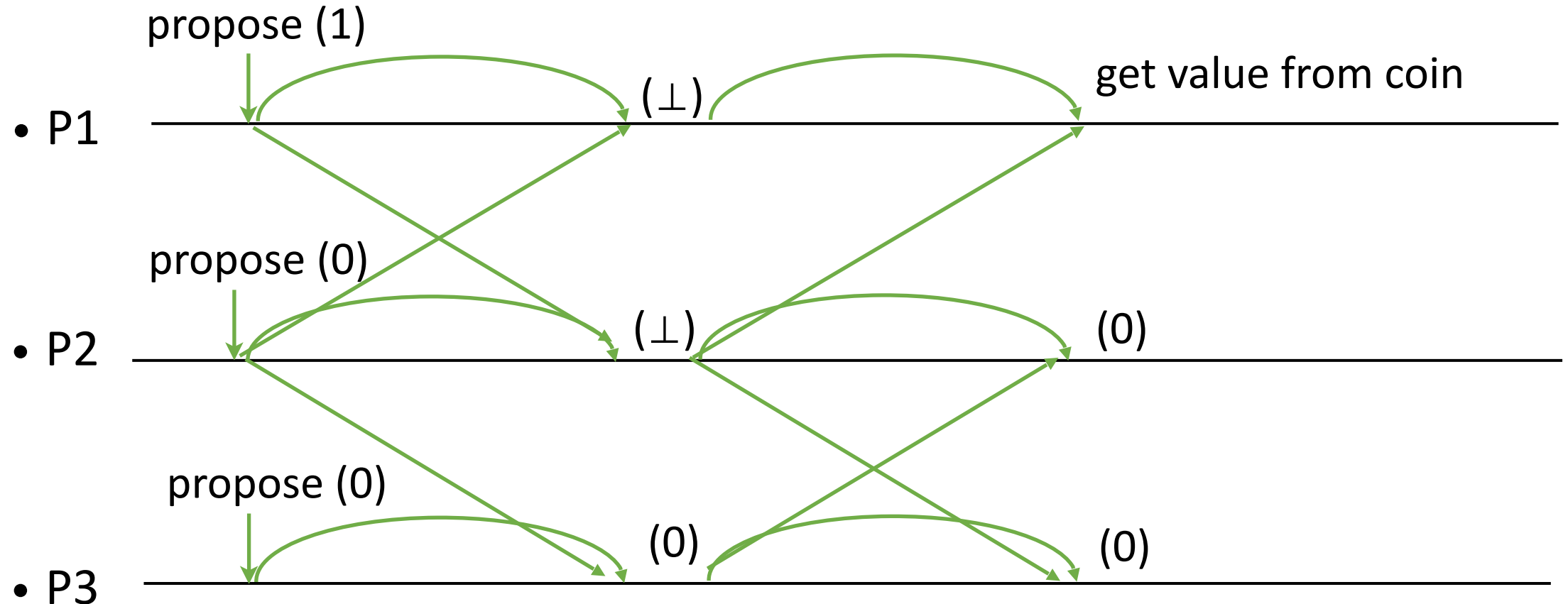
If not received non- $\perp$ value, get value from the coin for the next round



If not received non- $\perp$ value, get value from the coin for the next round

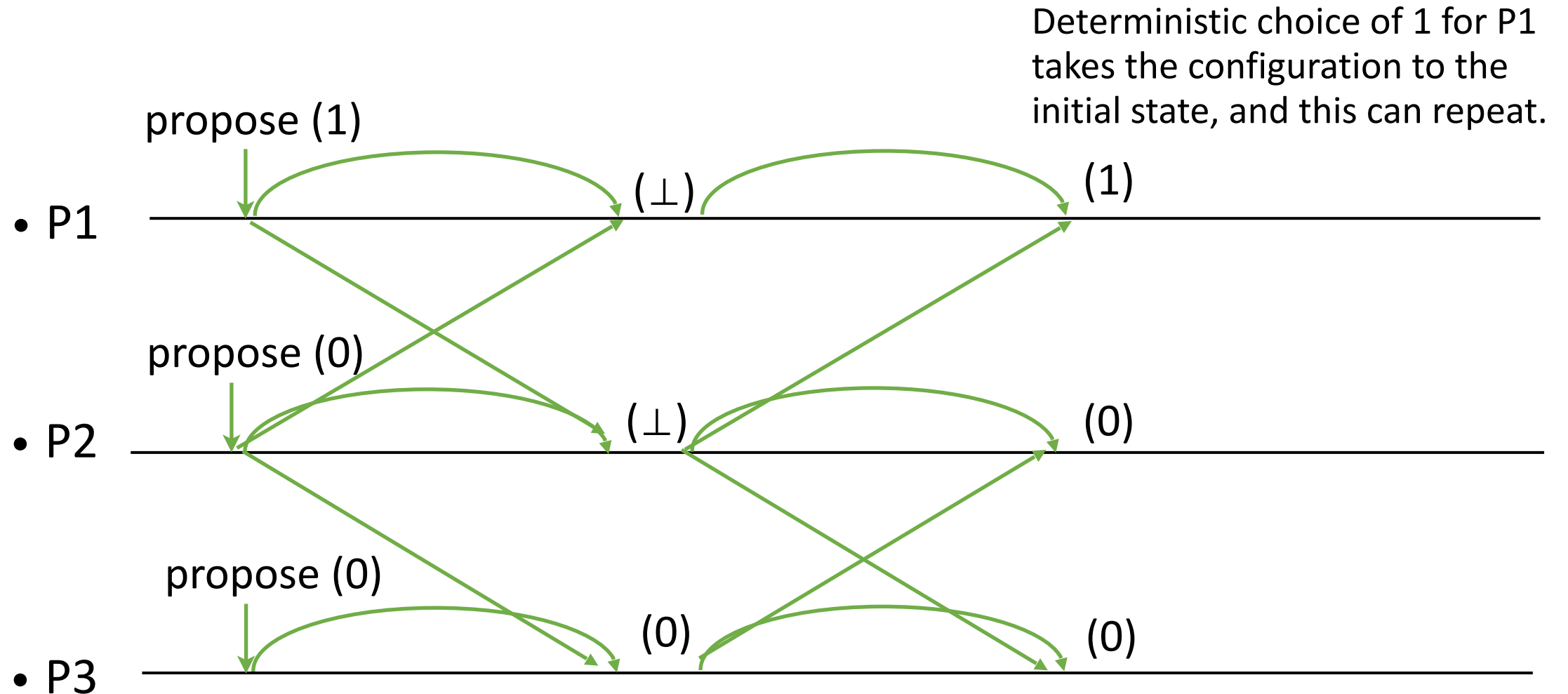


If not received non- $\perp$ value, get value from the coin for the next round



In Phase 2, $\perp$ is sent only when there are different proposals. So any binary value from the coin is valid.

Deterministic choice would lead to non-termination



Randomized Binary Consensus (1/3)

Implements: RandomizedConsensus, with domain $\{0, 1\}$.

Uses: BestEffortBroadcast, instance beb;
CommonCoin (multiple instances).

upon event $\langle \text{Init} \rangle$ **do**

round := 0; phase := 0; proposal := $\perp$; decided := $\perp$;

val := $[\perp]^n$

upon event $\langle \text{propose}(v) \rangle$ **do**

proposal := v; round := 1; phase := 1

trigger $\langle \text{beb, broadcast (Phase1(round, proposal))} \rangle$

Randomized Binary Consensus (2/3)

```
upon event  $\langle \text{beb}, \text{deliver}(p, \text{Phase1}(r, v)) \rangle$  such that  $\text{phase} = 1 \wedge r = \text{round}$  do  
   $\text{val}[p] := v$   
  if  $\text{decided} \neq \perp$  then  
    trigger  $\langle \text{beb}, \text{broadcast}(\text{Phase2}(\text{round}, \text{decided})) \rangle$   
  else if  $\#(\text{val}) > n/2$  do  
    if  $\exists v$  such that  $\text{val}[p]$  for all  $p$  then  
       $\text{proposal} := v$   
    else  
       $\text{proposal} := \perp$   
     $\text{val} := [\perp]^n; \text{phase} := 2$   
    trigger  $\langle \text{beb}, \text{broadcast}(\text{Phase2}(\text{round}, \text{proposal})) \rangle$ 
```

Randomized Binary Consensus (3/3)

```
upon event  $\langle \text{beb}, \text{deliver}(p, \text{Phase2}(r, v)) \rangle$  such that  $\text{phase} = 2 \wedge r = \text{round}$  do  
   $\text{val}[p] := v$   
  if  $\#(\text{val}) \geq n/2 \wedge \text{coin}[\text{round}]$  is not initialized then  
    Initialize a new instance  $\text{coin}[\text{round}]$  of CommonCoin with domain  $\{0, 1\}$   
    trigger  $\langle \text{coin}[\text{round}], \text{release} \rangle$   
upon event  $\langle \text{coin}[\text{round}], \text{output}(c) \rangle$  do  
  if  $\text{decided} \neq \perp$  then  
     $\text{proposal} := \text{decided}$   
  else if  $\exists v \neq \perp$  such that  $\#\{p \in \Pi \mid \text{val}[p] = v\} > n/2$  then  
     $\text{decided} := \text{proposal} := v$   
    trigger  $\langle \text{decide}(v) \rangle$   
  else if  $\exists p \in \Pi, v \neq \perp$  such that  $\text{val}[p] = v$  then  
     $\text{proposal} := v$   
  else  
     $\text{proposal} := c$   
 $\text{val} := [\perp]^n$ ;  $\text{round} := \text{round} + 1$ ;  $\text{phase} := 1$   
trigger  $\langle \text{beb}, \text{broadcast}(\text{Phase1}(\text{round}, \text{proposal})) \rangle$ 
```

Add code to stop one round after decision.
Optimization: After decision, send the value to others by a reliable broadcast and stop.

Proof of Properties

- **Agreement:**

Case 1. Decisions in the same round: because of quorum intersection in phase 1.

Case 2. Decisions in two rounds: The value decided in the first round is adopted by all correct processes for the second round.

- **Integrity:**

The handler issues decision only if no decision is already made.

- **Validity:**

A decided value in a round was proposed by a majority in that round. We show by induction on the rounds that a proposal in every round is a proposal from the first round. A process adopts its proposal for the next round either from this round which is proposed by a majority in this round, or from the coin. A value is taken from the coin only when the process receives $\perp$ from a majority in phase 2. A process sends $\perp$ for phase 2 after receiving different binary proposals in phase 1. So any binary value from the coin is a proposed value.

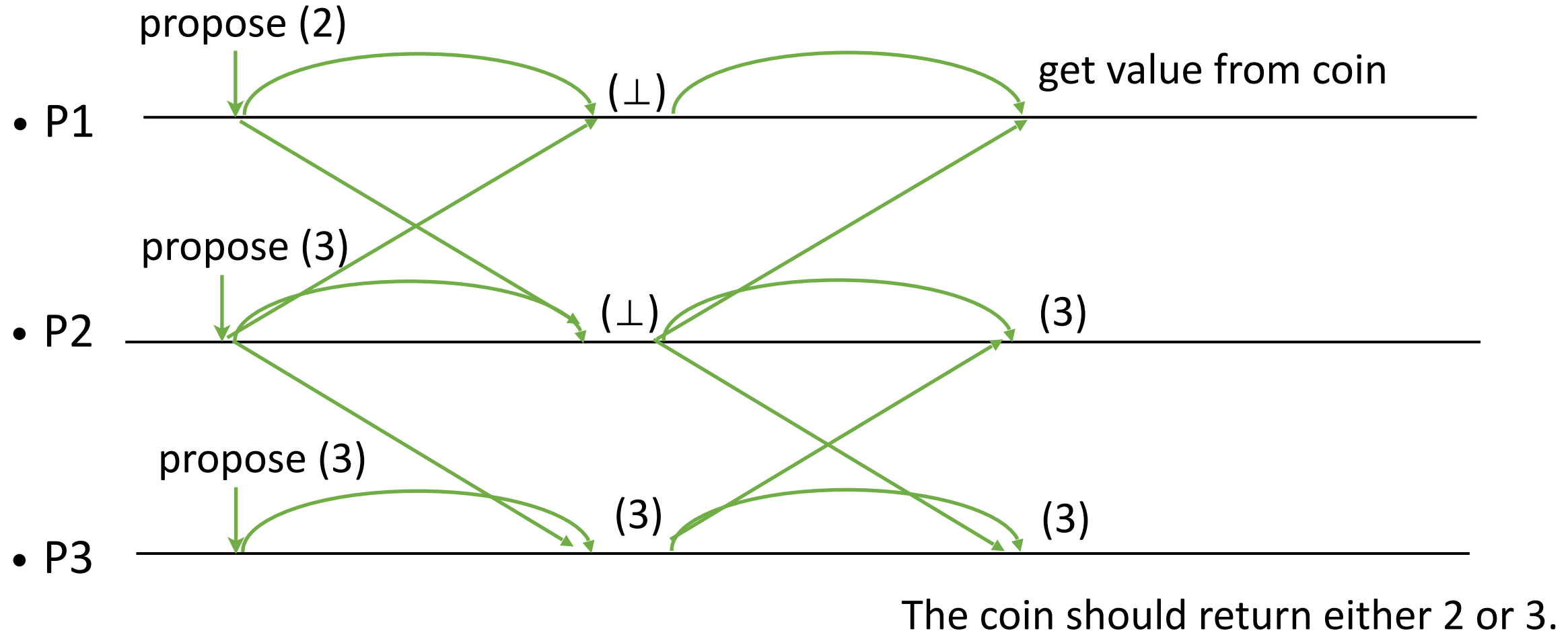
- **Probabilistic Termination:**

The proposals adopted from the previous round are the same because of quorum intersection in phase 1. The coin takes values uniformly at random. It will eventually match the adopted value if any, and all processes decide in that round.

Randomized Consensus for General Domains

- Common coin with a domain
- In order to satisfy **validity**, the coin should output only proposed values. The domain of the coin should be a subset of **proposed values**.
- In each round, each process broadcasts its proposal.
- Processes collect the set of proposals, and instantiate the coin with it.
- This set grows and processes eventually converge.
- Correct processes eventually invoke the coin with the same set.
- In the meanwhile, the coin provides termination so that the protocol makes progress.

Get value from the coin for the next round



Randomized Consensus for General Domains

Implements: RandomizedConsensus.

Uses: ...; ReliableBroadcast, instance rb

upon event $\langle \text{Init} \rangle$ **do**

...

values := $\emptyset$

upon event $\langle \text{propose}(v) \rangle$ **do**

...

values := values $\cup \{v\}$

trigger $\langle \text{rb}, \text{broadcast}(\text{Proposal}(v)) \rangle$

upon event $\langle \text{rb}, \text{deliver}(p, \text{Proposal}(v)) \rangle$ **do**

values := values $\cup \{v\}$;

upon $\#(\text{val}) > N/2 \wedge \text{phase} = 2 \wedge \text{decision} = \perp$ **do**

Initialize a new instance coin[round] of CommonCoin with domain values;

...